Master program
” **Advanced Analytics for Business**”

Advanced Visual Text Analytics– Natural Language Processing

Universitatea Națională de Știință si Tehnologie Politehnica București
Facultatea de Automatică și Calculatoare

Course 2

Logistic Regression for Sentiment Analysis

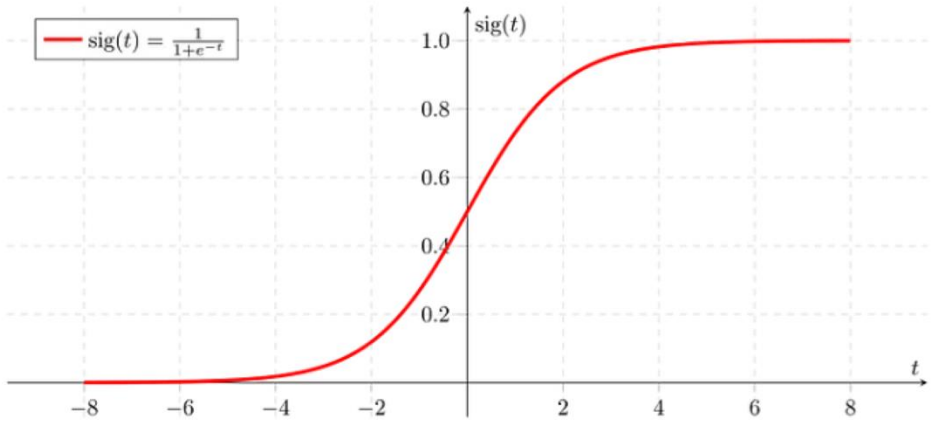
Lecture content

- What is *Logistic Regression*?
  - Logistic regression overview.
  - Training.
  - Cost function.
  - Gradient descent.
  - Testing.
- What are the steps for *Sentiment Analysis* using *Logistic regression*?
  - Vocabulary.
  - Feature extraction.
  - Preprocessing.
  - Algorithm structure.
- Python functions.

Logistic regression overview

- Logistic regression estimates the probability of an event occurring.
- Since the outcome is a probability, the dependent variable (output) is bounded between 0 and 1.
- Logistic regression makes use of the sigmoid function which outputs this probability between 0 and 1:

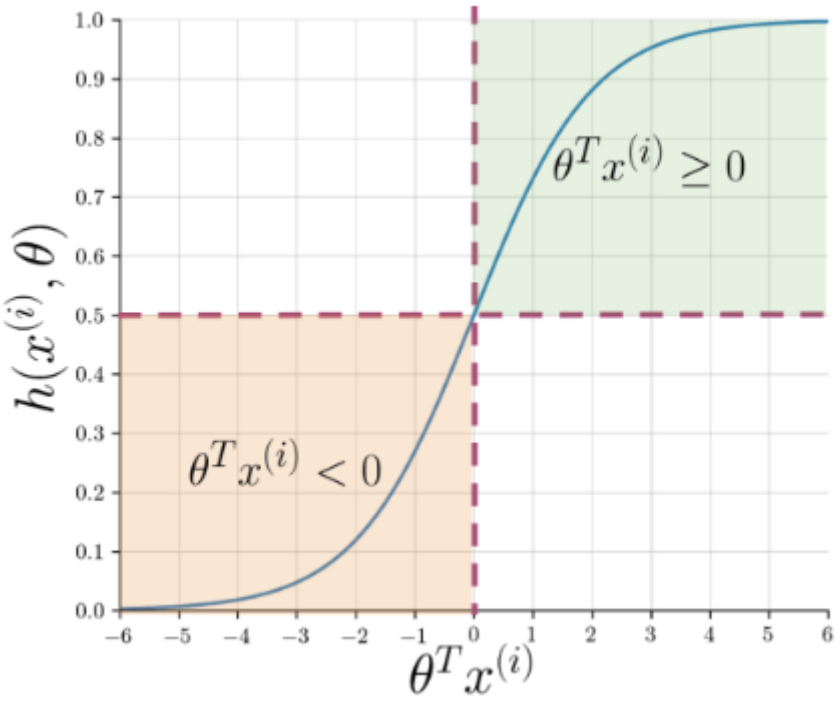
$$h\left(x^{(i)}, \theta\right) = \frac{1}{1+e^{-\theta^T x^{(i)}}},$$



θ – weight parameter;

$x^{(i)}$ – *input vector*;

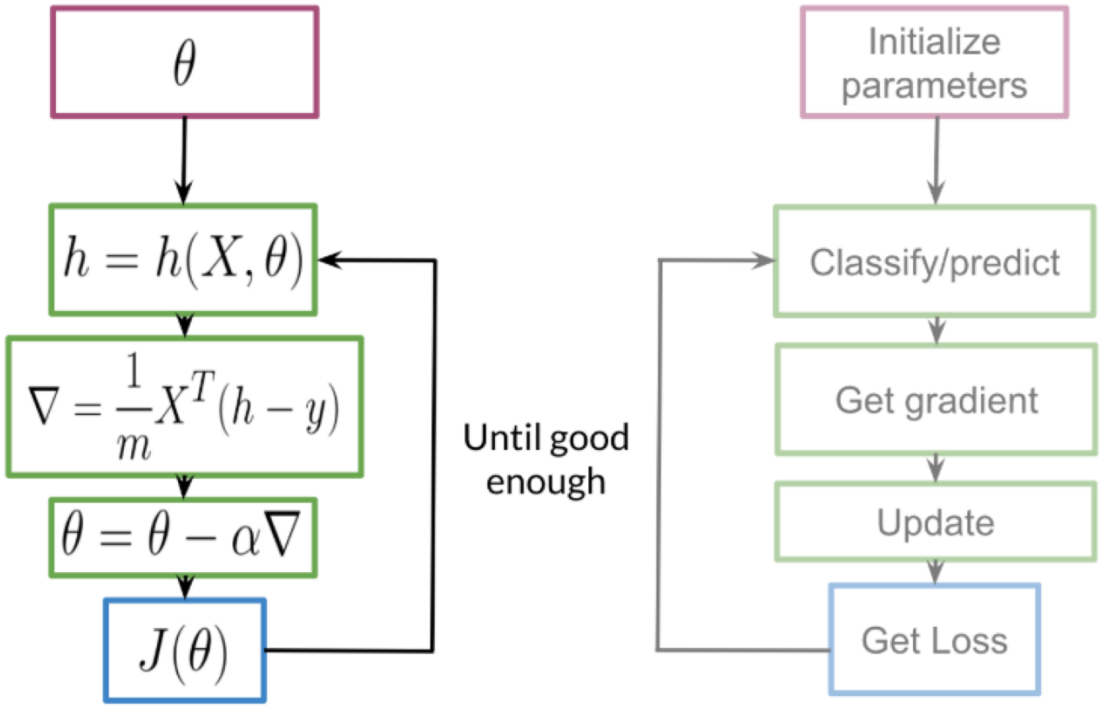
Logistic regression output interpretation



Example: $x^{(i)} = \begin{bmatrix} 1 \\ 238 \\ 108 \end{bmatrix}, \theta = \begin{bmatrix} 0.023 \\ 0.0042 \\ -0.0194 \end{bmatrix} \Rightarrow z = \theta^T x^{(i)} = 7.92,\ sig(z) = \frac{1}{1+0.00036} \approx 1.$

Logistic regression - Training

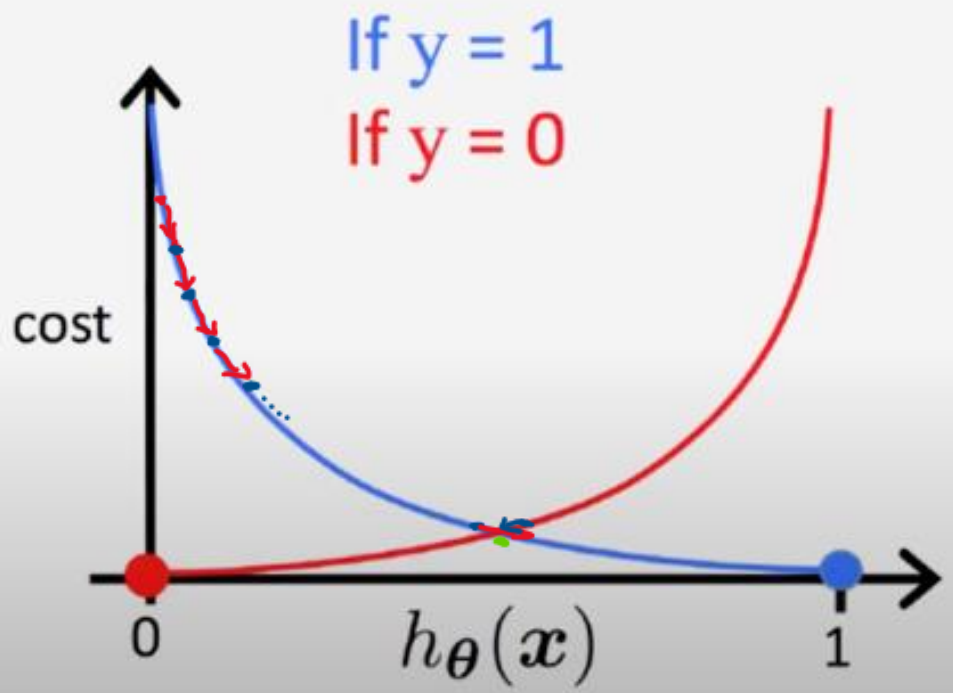
Steps of training process:



DeepLearning.AI

Logistic regression - Training

Convergence to optimum weights:



$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m} Cost(\, h_\theta(x^{(i)}), y^{(i)})$$

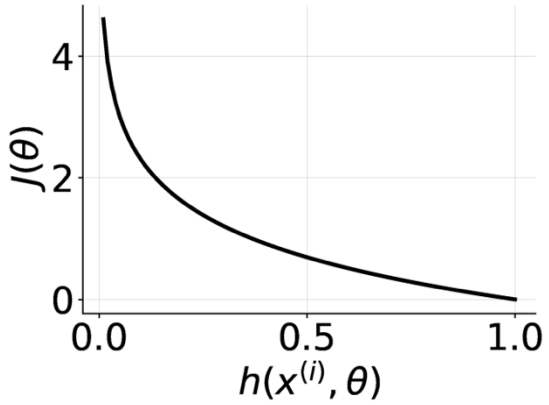
$$Cost(h_\theta(x)), y) = -\log(h_\theta(x)) \quad if\ y = 1$$
$$Cost(h_\theta(x)), y) = -\log(1 - h_\theta(x)) \quad if\ y = 0$$

Logistic regression – Cost function *(J)*

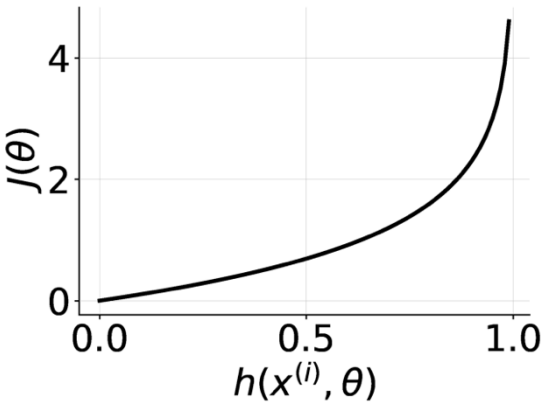
Binary cross-entropy function:

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(h\left(x^{(i)},\theta\right)\right) + \left(1-y^{(i)}\right)\log\left(1-h\left(x^{(i)},\theta\right)\right)\right]$$

- if $y^{(i)} = 0$, and $h\left(x^{(i)},\theta\right)$ = any then 0
- if $y^{(i)} = 1$, and $h\left(x^{(i)},\theta\right) = 0.999$ then 0
- if $y^{(i)} = 1$, and $h\left(x^{(i)},\theta\right) = 0.001$ then –inf



- if $y^{(i)} = 1$, and $h\left(x^{(i)},\theta\right)$ = any then 0
- if $y^{(i)} = 0$, and $h\left(x^{(i)},\theta\right) = 0.0001$ then 0
- if $y^{(i)} = 0$, and $h\left(x^{(i)},\theta\right) = 0.9999$ then -inf



Logistic regression – Gradient Descent

Given the cost function J:

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(h\left(x^{(i)},\theta\right)\right) + \left(1-y^{(i)}\right)\log\left(1-h\left(x^{(i)},\theta\right)\right)\right]$$

- the gradient descent is defined:

  Repeat{

  $$\theta_j := \theta_j - \alpha\frac{\partial}{\partial\theta_j}J(\theta), \text{ by calculation we obtain}$$

  $$\theta_j := \theta_j - \frac{1}{m}\sum_{1}^{m}\left[h\left(x^{(i)},\theta\right) - y^{(i)}\right]x_j^{(i)}$$

  } update each parameter simultaneously.

- Loop will repeat until global minimum will be found, or reduction of cost function will be very low.

Logistic regression – Testing

Given a *test set* defined by:

$$(X_{val}, Y_{val})$$

and parameters θ estimated previously by gradient descent algorithm, we'll predict the expected output:

$$pred = h(X_{val}, \theta).$$

and define the *accuracy* as:

$$\sum_1^m \frac{(pred^i == y_{val}^i)}{m}$$

Example:

$$Y_{val} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 1 \\ 1 \end{bmatrix}, pred = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \; acc = \frac{5}{6} = 0.833$$

Sentiment Analysis using LR

Steps that should be followed to perform Sentiment Analysis using LR:

1. Connect to data source (e.g. database, data lake, etc).
2. Build vocabulary: given a list of texts, *vocabulary V* will represent the list of unique words from this list.
3. Feature extraction with frequency (negative and positive counts).
4. Text preprocessing:
    - remove stop words, punctuation, urls.
    - Stemming and lowercasing.
5. Build training and test data sets (1-4).
6. LR Training.
    - define cost function.
    - parameter optimization using gradient descent.
7. LR Testing – check the accuracy.

1. Connect to data source

There are multiple sources & types of data used in Sentiment Analysis:

- Tweets.
- Messages, reviews.
- Feedback questionnaire.

Data can reside in diverse data structures:

- Unstructured format as field in a table/database.
- Unstructured format as noSQL DB (e.g. Mongo DB Atlas).
- Unstructured format as files (xml, json, parquet, etc) in Data Lake (e.g. AWS S3).

! Important: Data sources must be reliable, trustworthy (avoid bias).

2. Build vocabulary

First intuition is to build the vocabulary V from a various group of texts. This will create a huge vocabulary size (the number of all unique words).

Then for one given text for example:

I am delighted to develop deep learning algorithms.

the correspondent vector will be:

$$x = [1, 1, 0, 0, 1, 0, 0, \dots, 0, 0, 0, 1, 0, 0, 1], where\ n = |V|\ (length\ of\ V)$$

Sparse representation

! Chanllenges:

- Large parameter vector θ to be trained:

$$\theta = [\theta_0, \theta_1, \theta_2, \dots, \theta_n]$$

- Higher training cost & higher prediction/inference cost.

What to do in this case ??

3. Feature extraction with frequency (positive vs negative)

For a *balanced* set of positive and negative texts like:

Positive:

I am delighted to develop deep learning algorithms.

I am happy.

And negative*:*

I am unhappy, as I cannot improve my programming skills.

I feel awful.

We'll represent each text as a vector of dimension 3:

$$X_m = [1, \sum_w freq(w, 1), \sum_w freq(w, 0)]$$

Example:

I am happy learning programming algorithms.

x = [1, 7, 5] - *?? Confusing – then we need to exclude unmeaningful words.*

vocabulary	Positive Freq	Negative Freq
I	2	3
am	2	1
delighted	1	0
develop	1	0
deep	1	0
learning	1	0
algorithms	1	0
happy	1	0
unhappy	0	1
cannot	0	1
improve	0	1
programming	0	1
skills	0	1
feel	0	1
awful	0	1

4. Text preprocessing

When preprocessing we have to perform the following actions:

- *eliminate handles (@gabriel.florea), URLs.*
- *tokenize the text/string into words.*
- *remove stop words: I, am, is on, and, if, etc.*
- *stemming (convert every word to its stem).*
- *convert all words/tokens to lower case.*

Example:

You will receive model details from my email address gabriel.florea@university.com

Eliminate handles, email address, URLs:

You will receive model details from my email address

Tokenize:

[You, will, receive, model, details, from, my, email, address]

4. Text preprocessing

Remove stop words:

[receive, model, details, email, address]

Stemming:

[receiv, model, detail, email, address]

Lower casing:

[receiv, model, detail, email, address].

Get vector frequencies

$$\boldsymbol{X} = \begin{bmatrix} 1 & X_1^{(1)} & X_2^{(1)} \\ 1 & X_1^{(2)} & X_2^{(2)} \\ \vdots & \vdots & \vdots \\ 1 & X_1^{(m)} & X_2^{(m)} \end{bmatrix}$$

Bias term = 1 is used as the log-odds of the positive class and also for model flexibility.

Python functions

- Import library NLTK:
  - NLTK –Natural Languge Toolkit - a leading platform for building Python programs to work with human language data. Ease of use and understanding: https://www.nltk.org/book/.
  - Text Corpora and Lexical Resources:

```
import nltk
nltk.download()
```

```
showing info https://raw.githubusercontent.com/nltk/nltk_data/gh-pages/index.xml
2025-01-01 20:30:49.203 python[26036:1591187] +[IMKClient subclass]: chose IMKClient_Modern
2025-01-01 20:30:49.203 python[26036:1591187] +[IMKInputSession subclass]: chose IMKInputSession_Modern
True
```

  - Examples of samples data for Sentiment Analysis:
    - movie_review.
    - pro_cons.
    - twitter_samples.

! It is your choice to use whatever dataset you want. Analyze and document it first.

Python functions

- Import library NLTK:
  - *PorterStemmer*- stemming is the process of producing morphological variants of a root/base word. Ex: *Porter Stemmer* available in nltk library.
  - Tokenize – divide strings into list of substrings. Can be used to find words and punctuation in a string.
    - E.g.:

Sentence tokenization in word_tokenize:

```
>>> s11 = "I called Dr. Jones. I called Dr. Jones."
>>> word_tokenize(s11)
['I', 'called', 'Dr.', 'Jones', '.', 'I', 'called', 'Dr.', 'Jones', '.']
>>> s12 = ("Ich muss unbedingt daran denken, Mehl, usw. fur einen "
...        "Kuchen einzukaufen. Ich muss.")
>>> word_tokenize(s12)
['Ich', 'muss', 'unbedingt', 'daran', 'denken', ',', 'Mehl', ',', 'usw',
 '.', 'fur', 'einen', 'Kuchen', 'einzukaufen', '.', 'Ich', 'muss', '.']
```

- *Stopwords*: - a set of commonly used words (e.g.: “the”, “is”, “and”, “when” etc).

```
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.tokenize import TweetTokenizer
```

Python functions

- Import library *np*:
  - NumPy –offers comprehensive mathematical functions, linear algebra etc. Ease of use and understanding: https://numpy.org/.
- Import library *pandas*:
  - Pandas is a fast powerful, flexible and easy of use open source data analysis and manipulation tool: https://pandas.pydata.org/

```
import numpy as np
import pandas as pd
from nltk.corpus import movie_reviews
```

- Auxiliary library *os:*
  - *Miscellaneous operating system interfaces – portable way of using OS dependent functionality (read-write files, manipulate paths, create temp files etc).*
  - Example: `from os import getcwd`

Python functions

- Build a function for *text processing*:
  - Inputs: a string/text;
  - Outputs: a list of "clean" words containing initial text;
  - Initialize a PorterStemmer;
  - Initialize a *stopword* list – e.g. English language;
  - Use a series of methods to remove special characters;
    - using *regular expressions* remove hashtags, hyperlinks, email addresses, etc;
  - Initialize a tokenizer and then create the list of *tokens;*
  - Create a *loop* parsing each token from the list in which:
    - Remove stop words;
    - Remove punctuation;
    - Use stemming;
  - Return the list of "clean" words/tokens.

Python functions

```
def process_text(str):
    """Process text function.
    Inputs:
        text: a string containing a text
    Output:
        tokens_clean: a list of words containing the processed string

    """
    stemmer = PorterStemmer()
    stopwords_english = stopwords.words('english')
    # remove hyperlinks
    str = re.sub(r'https?://[^\s\n\r]+', '', str)
    # remove hashtags
    # only removing the hash # sign from the word
    str = re.sub(r'#', '', str)
    # tokenize tweets
    tokenizer = TweetTokenizer(preserve_case=False, strip_handles=True,
                               reduce_len=True)
    str_tokens = tokenizer.tokenize(str)

    tokens_clean = []
    for word in str_tokens:
        if (word not in stopwords_english and  # remove stopwords
                word not in string.punctuation):  # remove punctuation
            stem_word = stemmer.stem(word)  # stemming word
            tokens_clean.append(stem_word)

    return tokens_clean
```

Python functions

- Build a function for *frequency* of positive and negative sentiments:
  - Inputs: a list of strings/texts, array of labels/sentiments (0 or 1);
  - Outputs: a dictionary of pairs of each word and label;

```
def build_freqs(strs, ys):
    """
    Inputs: strs: a list of strings
            ys: an m x 1 array with the label (0 or 1) of each string
    Outputs:
        freqs: a dictionary of pairs of each word and label
    """
    yslist = np.squeeze(ys).tolist()
    # Initialize an empty dictionary and populate it by looping over all strings
    freqs = {}
    for y, str in zip(yslist, strs):
        for word in process_text(str):
            pair = (word, y)
            if pair in freqs:
                freqs[pair] += 1
            else:
                freqs[pair] = 1

    return freqs
```

Python functions

- Load data positive and negative – *balanced classes*:

```
[11]: # read the pros and cons reviews
reviews_cons = pros_cons.raw('IntegratedCons.txt')
reviews_pros = pros_cons.raw('IntegratedPros.txt')
reviews_pros
```

```
[11]: '       <Pros>Easy to use, economical!</Pros>\n       <Pros>Digital is where it\'s at...down with developing film!</Pros>\n       <Pros
>Good image quality, 3x optical zoom, macro mode, inexpensive</Pros>\n       <Pros>Awesome features/easy to use/fun/versatile/low price/C
ust SVS 2nd 2 none!</Pros>\n       <Pros>intuitive, user friendly</Pros>\n       <Pros>Simple, Flexable, Reliable</Pros>\n       <Pros>
battery life, download speed</Pros>\n       <Pros>Agfa quality in a small box, great for table top and tests</Pros>\n       <Pros>Image
quality, price, video link</Pros>\n       <Pros>No Extras to Buy, Very Flexible, High Quality Images, Good Software</Pros>\n       <Pros
>simplicity</Pros>\n       <Pros>Comes with software, good price, excellent pictures.</Pros>\n       <Pros>Clear pics</Pros>\n       <P
ros>Price, Over all Performance, Zoom Feature</Pros>\n       <Pros>clear, srisp picture, durable camera, good desing, nice features</Pros
>\n       <Pros>Inexpensive compared to some</Pros>\n       <Pros>Great picture quality, swivel lens, many options</Pros>\n       <Pros
>Very simple controls and a wide range of features</Pros>\n       <Pros>Great camera, easy to use, good looking, high quality images.</Pr
os>\n       <Pros>Megapixel quality, easy to use</Pros>\n       <Pros>durable, easy to use</Pros>\n       <Pros>Sharp, crisp, pictures.
</Pros>\n       <Pros>Ease of use, quality pictures.</Pros>\n       <Pros>pic quality, smart media,price</Pros>\n       <Pros>megapixel
quality, easy to use</Pros>\n       <Pros>Easy to use, takes GREAT pictures, full of features.</Pros>\n       <Pros>Saturated, sharp ima
ges at all resolutions, twist-style body</Pros>\n       <Pros>very clear photos</Pros>\n       <Pros>Nickle Hydride batteries,  ease of
use, excellent color rendition</Pros>\n       <Pros>Picture quality, Twist body</Pros>\n       <Pros>perfect shots, long-focus, easy to
use, a lot of features, good software</Pros>\n       <Pros>Easy to use, stylish, good picture quality, good price, good company.</Pros>\n
<Pros>Good quality pictures for a low price, great first camera</Pros>\n       <Pros>This camera is a nice easy to use camera.</Pros>\n
<Pros>Takes great pictures</Pros>\n       <Pros>Inexpensive and clear pictures</Pros>\n       <Pros>Camera is very easy to learn and us
e.</Pros>\n       <Pros>extra film, led for image viewing, software is easy to use, sleek design</Pros>\n       <Pros>Easy to use, great
```

- split *80% train* and *20% test* data sets

```
#remove begin,end tags <Pros>, <Cons>
reviews_cons = reviews_cons.replace('<Cons>','').replace('</Cons>','').strip()
reviews_pros = reviews_pros.replace('<Pros>','').replace('</Pros>','').strip()
reviews_cons_list = reviews_cons.split('\n')
reviews_pros_list = reviews_pros.split('\n')
```

Python functions

- Training and test sets are:

[105]:
```
train_pos = reviews_pros_list[:16000]
test_pos = reviews_pros_list[16000:20000]
train_neg = reviews_cons_list[:16000]
test_neg = reviews_cons_list[16000:20000]
train_x = train_pos + train_neg
test_x = test_pos + test_neg
```

- Create the numpy array of positive labels and negative labels.

[108]:
```
# combine positive and negative labels
train_x = train_pos + train_neg
test_x = test_pos + test_neg
train_y = np.append(np.ones((len(train_pos), 1)), np.zeros((len(train_neg), 1)), axis=0)
test_y = np.append(np.ones((len(test_pos), 1)), np.zeros((len(test_neg), 1)), axis=0)
```

[110]:
```
# Print the shape train and test sets
print("train_y.shape = " + str(train_y.shape))
print("test_y.shape = " + str(test_y.shape))
```

```
train_y.shape = (32000, 1)
test_y.shape = (8000, 1)
```

- Finally apply *process_text* and *build_freqs* functions.

Python functions

- Sigmoid function:

- The sigmoid function is defined as:

$$h(z) = \frac{1}{1 + \exp^{-z}}$$

It maps the input 'z' to a value that ranges between 0 and 1, and so it can be treated as a probability.

How to build a python function for sigmoid function??

DeepLearning.AI

Python functions

- Sigmoid function:

```
def sigmoid(z):
    '''
    Inputs:
        z: is the input (can be a scalar or an array)
    Outputs:
        h: the sigmoid of z
    '''
    # calculate the sigmoid of z
    h = 1/(1+np.exp(-z))

    return h
```

Python functions

- Gradient descent function – define cost function:

The cost function used for logistic regression is the average of the log loss across all training examples:

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m} y^{(i)} \log(h(z(\theta)^{(i)})) + (1 - y^{(i)}) \log(1 - h(z(\theta)^{(i)}))$$

- m is the number of training examples
- $y^{(i)}$ is the actual label of training example 'i'.
- $h(z^{(i)})$ is the model's prediction for the training example 'i'.

The loss function for a single training example is

$$Loss = -1 \times \left(y^{(i)} \log(h(z(\theta)^{(i)})) + (1 - y^{(i)}) \log(1 - h(z(\theta)^{(i)}))\right)$$

- All the h values are between 0 and 1, so the logs will be negative. That is the reason for the factor of -1 applied to the sum of the two loss terms.
- Note that when the model predicts 1 ($h(z(\theta)) = 1$) and the label 'y' is also 1, the loss for that training example is 0.
- Similarly, when the model predicts 0 ($h(z(\theta)) = 0$) and the actual label is also 0, the loss for that training example is 0.
- However, when the model prediction is close to 1 ($h(z(\theta)) = 0.9999$) and the label is 0, the second term of the log loss becomes a large negative number, which is then multiplied by the overall factor of -1 to convert it to a positive loss value. $-1 \times (1 - 0) \times log(1 - 0.9999) \approx 9.2$ The closer the model prediction gets to 1, the larger the loss.

DeepLearning.AI

Python functions

- Gradient descent function – update weights vector (model):

To update your weight vector θ, you will apply gradient descent to iteratively improve your model's predictions.
The gradient of the cost function J with respect to one of the weights θ_j is:

$$\nabla_{\theta_j} J(\theta) = \frac{1}{m} \sum_{i=1}^{m} (h^{(i)} - y^{(i)}) x_j^{(i)}$$

- 'i' is the index across all 'm' training examples.
- 'j' is the index of the weight θ_j, so $x_j^{(i)}$ is the feature associated with weight θ_j
- To update the weight θ_j, we adjust it by subtracting a fraction of the gradient determined by α:

$$\theta_j = \theta_j - \alpha \times \nabla_{\theta_j} J(\theta)$$

- The learning rate α is a value that we choose to control how big a single update will be.

DeepLearning.AI

Python functions

- Gradient descent function – implement gradient descent function:

  - The number of iterations 'num_iters" is the number of times that you'll use the entire training set.
  - For each iteration, you'll calculate the cost function using all training examples (there are 'm' training examples), and for all features.
  - Instead of updating a single weight θ_i at a time, we can update all the weights in the column vector:

$$\theta = \begin{pmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \theta_n \end{pmatrix}$$

  - θ has dimensions (n+1, 1), where 'n' is the number of features, and there is one more element for the bias term θ_0 (note that the corresponding feature value $\mathbf{x_0}$ is 1).
  - The 'logits', 'z', are calculated by multiplying the feature matrix 'x' with the weight vector 'theta'. $z = \mathbf{x}\theta$
    - $\mathbf{x}$ has dimensions (m, n+1)
    - θ: has dimensions (n+1, 1)
    - $\mathbf{z}$: has dimensions (m, 1)
  - The prediction 'h', is calculated by applying the sigmoid to each element in 'z': $h(z) = sigmoid(z)$, and has dimensions (m,1).
  - The cost function J is calculated by taking the dot product of the vectors 'y' and 'log(h)'. Since both 'y' and 'h' are column vectors (m,1), transpose the vector to the left, so that matrix multiplication of a row vector with column vector performs the dot product.

$$J = \frac{-1}{m} \times \left(\mathbf{y}^T \cdot log(\mathbf{h}) + (\mathbf{1} - \mathbf{y})^T \cdot log(\mathbf{1} - \mathbf{h}) \right)$$

  - The update of theta is also vectorized. Because the dimensions of $\mathbf{x}$ are (m, n+1), and both $\mathbf{h}$ and $\mathbf{y}$ are (m, 1), we need to transpose the $\mathbf{x}$ and place it on the left in order to perform matrix multiplication, which then yields the (n+1, 1) answer we need:

$$\theta = \theta - \frac{\alpha}{m} \times \left(\mathbf{x}^T \cdot (\mathbf{h} - \mathbf{y}) \right)$$

DeepLearning.AI

Python functions

- Gradient descent function – implement gradient descent function (python):

```
def gradientDescent(x, y, theta, alpha, num_iters):
    '''
    Inputs:
        x: matrix of features which is (m,n+1)
        y: corresponding labels of the input matrix x, dimensions (m,1)
        theta: weight vector of dimension (n+1,1)
        alpha: learning rate
        num_iters: number of iterations you want to train your model for
    Outputs:
        J: the final cost
        theta: your final weight vector
    '''
    # get 'm', the number of rows in matrix x
    m = len(x)
    for i in range(0, num_iters):

        # get z, the dot product of x and theta
        z = ......

        # get the sigmoid of z
        h = ......

        # calculate the cost function
        J = ......

        # update the weights theta
        theta = ......
    J = float(J)
    return J, theta
```

Python functions

- Gradient descent function – test gradient descent function (python):

```
# Check the gradient descent function
np.random.seed(1)
# X input is 20 x 3 with ones for the bias terms
tmp_X = np.append(np.ones((20, 1)), np.random.rand(20, 2) * 2000, axis=1)
# Y Labels are 20 x 1
tmp_Y = (np.random.rand(20, 1) > 0.35).astype(float)

# Apply gradient descent
tmp_J, tmp_theta = gradientDescent(tmp_X, tmp_Y, np.zeros((3, 1)), 1e-8, 1000)
# you can check gradient descent performance with different number of iterations (300, 500, 700, 1000, 1200, 1500)
print(f"The cost after training is {tmp_J:.8f}.")
print(f"The resulting vector of weights is {[round(t, 8) for t in np.squeeze(tmp_theta)]}")
```

Python functions

- Put all steps together and build the Sentiment Analysis algorithm using Logistic Regression
  - Connect to data;
  - Choose data set;
  - Prepare data (training and testing data set);
  - Data/Text processing;
  - Build frequencies and features matrix;
  - Build sigmoid function;
  - Build gradient descent function;
  - Train model;
  - Test model;
  - Assess performance/ accuracy;
  - Conclusions;

Good luck and enjoy ☺